Functions

Computer Programming 1 · Lecture 6

Siyavushkhon Kholmatov · New Uzbekistan University

Week 6 · Fall 2026

Where we are

Last week

The three loops, nested loops — and the first functions: a return type, a parameter, a body, a `return`.

Today

Functions properly: how arguments travel, how one name can serve several functions, and the difference between handing over a *copy* and handing over the *variable itself*.

Next week

Putting functions to work in the lab and revision.

The one idea of this lecture

A function call is a contract: the caller supplies values of agreed types, the function returns a value of an agreed type.

The shape of a function

The four parts

```
int square(int x) {
    return x * x;
}
```

Piece	Here	What it means
return type	`int`	the type of the value handed back
name	`square`	what the caller writes
parameter	`int x`	a variable that receives the caller's value
body	`{ ...}`	the work, run only when called

`return` does two things at once

It hands the value back *and* ends the function immediately — any lines after it in that branch never run.

Function declaration

Does not compile

```
int main() {
    cout << square(5);   // ERROR
}

int square(int x) {
    return x * x;
}
```

The compiler reads top to bottom. At the call it has never heard of `square`.

Compiles

```
int square(int x);       // declaration

int main() {
    cout << square(5);   // 25
}

int square(int x) {      // definition
    return x * x;
}
```

Declaration (prototype) vs definition

The declaration is the contract alone, ending in a semicolon: return type, name, parameter types. The definition is the contract plus the body. You may declare many times, but define exactly once.

Function declaration — cont.

```cpp
#include <iostream>
using namespace std;

double area(double r);        // prototypes
int    maxOf(int a, int b);

int main() {                  // the story
    cout << area(2) << endl;
    cout << maxOf(3, 9) << endl;
    return 0;
}

double area(double r) {       // the definition
    return 3.14159 * r * r;
}
int maxOf(int a, int b) {
    return (a > b) ? a : b;
}
```

Output

```
12.5664
9
```

Why this order

A reader meets `main` first and sees *what* the program does; the bodies below say *how*.

Parameter names in a prototype are optional — `double area(double);` is legal too — but keep them: they document the contract.

`void`: a function that does something instead of computing

```
void printLine(int n) {
    for (int i = 1; i <= n; i++) {
        cout << "-";
    }
    cout << endl;
}

void checkAge(int age) {
    if (age < 0) {
        cout << "Invalid" << endl;
        return;              // leave early
    }
    cout << "Age is " << age << endl;
}
```

`void` means "no value comes back"

You call it as a statement of its own: `printLine(20);`

You cannot write `int x = printLine(20);` — there is nothing to store.

`return;` with no value

Legal only in a `void` function. It is the way to stop early.

Your turn A

```
void greet() {
    cout << "Hi ";
}

int main() {
    cout << "A ";
    greet();
    cout << "B ";
    greet();
    cout << "C";
}
```

Your turn A

```
void greet() {
    cout << "Hi ";
}

int main() {
    cout << "A ";
    greet();
    cout << "B ";
    greet();
    cout << "C";
}
```

Output

```
A Hi B Hi C
```

`main` pauses at each call, runs the body, and carries on at the next line. Defining a function does not run it — calling it does.

Your turn B

```
int f(int x) {
    cout << "f ";
    return x + 1;
}

int main() {
    cout << f(2) + f(3);
}
```

Your turn B

```
int f(int x) {
    cout << "f ";
    return x + 1;
}

int main() {
    cout << f(2) + f(3);
}
```

Output

```
f f 7
```

Both calls run — and both print — *before* the addition can happen. Then $3 + 4$ is printed.

Built-in functions

Built-in or user-defined?

	Built-in (library)	User-defined
Who wrote it	the C++ standard library	you
How you get it	`#include` the header	write the definition
Examples	`sqrt`, `pow`, `round`, `setw`	`square`, `area`, `maxOf`
You can see the body	no, and you do not need to	yes

Both are called exactly the same way

`name(arguments)`. The compiler only needs the declaration — which is what an `#include` brings in.

`#include <cmath>` — example

Function	Meaning	Call	Value
`sqrt(x)`	square root	`sqrt(81)`	`9`
`pow(x, y)`	x^y	`pow(2, 10)`	`1024`
`fabs(x)`	absolute value	`fabs(-7.5)`	`7.5`
`floor(x)`	round down	`floor(7.8)`	`7`
`ceil(x)`	round up	`ceil(7.2)`	`8`
`round(x)`	nearest, halves go away from zero	`round(2.5)`	`3`

Build your own on top of theirs

```cpp
#include <iostream>
#include <cmath>
using namespace std;

double hypotenuse(double a, double b) {
    return sqrt(a * a + b * b);
}

int main() {
    cout << hypotenuse(3, 4) << endl;   // 5
    cout << hypotenuse(5, 12) << endl;  // 13
}
```

Layers

Your function is one line because somebody else wrote `sqrt`.

That is the whole idea: each layer hides the one below it.

Watch the argument types

`hypotenuse(3, 4)` passes two `int`s — each is converted to `double` at the call, because that is what the contract asks for.

Your turn A

```
#include <cmath>

cout << pow(2, 10) << " "
     << sqrt(81)   << " "
     << abs(-7);
```

Your turn A

```cpp
#include <cmath>

cout << pow(2, 10) << " "
     << sqrt(81)   << " "
     << abs(-7);
```

Output

```
1024 9 7
```

All three are `double` values with nothing after the point, so `cout` prints them without one.

Your turn B

```
#include <cmath>

cout << floor(7.8)   << " "
     << ceil(7.2)    << " "
     << round(2.5)   << " "
     << round(-2.5);
```

Your turn B

```cpp
#include <cmath>

cout << floor(7.8)   << " "
     << ceil(7.2)    << " "
     << round(2.5)   << " "
     << round(-2.5);
```

Output

```
7 8 3 -3
```

`floor` and `ceil` always go down and up. `round` sends a half *away* from zero — so -2.5 becomes -3, not -2.

Parameters and arguments

Two words that are easy to mix up

```
int maxOf(int a, int b) { ... }   // a and b are PARAMETERS

cout << maxOf(3, 9);              // 3 and 9 are ARGUMENTS
```

Parameter	a variable in the function's header; it exists while the call runs
Argument	the value the caller supplies at the call site

The matching rules

Same number, in the same order, with types that the compiler can convert. `maxOf(9, 3)` is a different call from `maxOf(3, 9)` — the order is the meaning.

Conversions happen at the call

```
double half(double x) {
    return x / 2;
}

int takeInt(int x) {
    return x;
}

cout << half(7)      << endl;
cout << takeInt(7.9) << endl;
```

Conversions happen at the call

```
double half(double x) {
    return x / 2;
}

int takeInt(int x) {
    return x;
}

cout << half(7)      << endl;
cout << takeInt(7.9) << endl;
```

Output

```
3.5
7
```

The `int` `7` widens to `7.0` — safe.
The `double` `7.9` narrows to `7` — the fraction is gone, silently.

The parameter type wins, always

Whatever you pass is converted to what the header asks for, by the same rules as an assignment.

Scope: what lives where

```
int triple(int n) {
    int result = 3 * n;     // local
    return result;
}

int main() {
    int n = 5;              // a different n
    cout << triple(n) << endl;
    // cout << result;      // ERROR: not here
    cout << n << endl;
}
```

Output

```
15
5
```

Two rules

A parameter or local variable exists only inside its own function, from its declaration to the closing brace.

Two functions may use the same name for different variables — they never meet.

Default arguments

```
double price(double base, double rate = 0.12) {
    return base * (1 + rate);
}

cout << price(100)       << endl;   // 112
cout << price(100, 0.5)  << endl;   // 150
```

The rule

Defaults fill in from the right. Once a parameter has one, every parameter after it must have one too.

Write the default in one place only

If the function has a prototype, the default goes *there* — repeating it on the definition is an error.

Use defaults for the common case: the caller who wants the usual 12% does not have to say so.

Your turn A

```
int add(int a, int b) {
    a = a + b;
    return a;
}

int main() {
    int x = 3, y = 4;
    cout << add(x, y) << " " << x;
}
```

Your turn A

```
int add(int a, int b) {
    a = a + b;
    return a;
}

int main() {
    int x = 3, y = 4;
    cout << add(x, y) << " " << x;
}
```

Output

```
7 3
```

`a` is the function's own variable. Changing it has no effect at all on `x` — which is the whole subject of the next section.

Your turn B

```
double price(double base,
             double rate = 0.12) {
    return base * (1 + rate);
}

int main() {
    cout << price(100) << " "
         << price(100, 0.5);
}
```

Your turn B

```
double price(double base,
             double rate = 0.12) {
    return base * (1 + rate);
}

int main() {
    cout << price(100) << " "
         << price(100, 0.5);
}
```

Output

```
112 150
```

The first call uses the default `0.12`; the second supplies its own rate and ignores it.

Call by value, call by reference

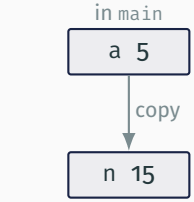
By value: the function gets a copy

```
void addTen(int n) {
    n += 10;
    cout << "inside: " << n << endl;
}

int main() {
    int a = 5;
    addTen(a);
    cout << "outside: " << a << endl;
}
```

Output

```
inside:  15
outside:  5
```



This is the default in C++: call by value. The argument is copied into the parameter, and the original cannot be touched.

Which is usually exactly what you want

```
int factorial(int n) {
    int result = 1;
    while (n > 1) {     // n is destroyed
        result *= n;
        n--;            // ... but it is OUR n
    }
    return result;
}

int main() {
    int k = 5;
    cout << factorial(k) << " " << k;
}
```

Output

```
120 5
```

The safety of a copy

The function may use its parameter as scratch paper — counting it down to 1 — and the caller's `k` is untouched.

The swap that does nothing

```
void swapValues(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 1, y = 2;
    swapValues(x, y);
    cout << x << " " << y;
}
```

The swap that does nothing

```
void swapValues(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 1, y = 2;
    swapValues(x, y);
    cout << x << " " << y;
}
```

Output

```
1 2
```

Nothing is wrong with the algorithm

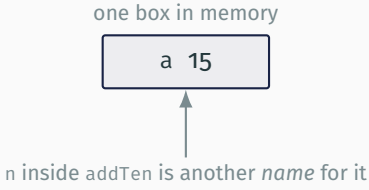
The three lines do swap — they swap the *copies*, which are thrown away when the function returns.

To change the caller's variables, the function must be given the variables, not their values.

By reference: one variable, two names

```
void addTen(int& n) {      // note the &
    n += 10;
}

int main() {
    int a = 5;
    addTen(a);
    cout << a << endl;     // 15
}
```



`int& n` means "`n` is a reference to an `int`"

No copy is made — every use of `n` inside the function reads and writes the caller's variable. The call site looks identical: `addTen(a);`, so you must read the header to know which kind it is.

The swap that works

```
void swapValues(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 1, y = 2;
    swapValues(x, y);
    cout << x << " " << y;
}
```

Output

```
2 1
```

Two characters

The only change is `&` twice in the header. The body, the call and the output of the program are otherwise identical.

`temp` is still an ordinary local copy — you need it, or the first assignment destroys `a`.

Giving back more than one answer

```
void split(int total, int& h, int& m) {
    h = total / 60;
    m = total % 60;
}

int main() {
    int hours, minutes;
    split(135, hours, minutes);
    cout << hours << " h " << minutes << " min";
}
```

Output

```
2 h 15 min
```

Why a reference here

`return` hands back exactly one value. When a function must produce two, the extra answers go out through reference parameters.

Notice `hours` and `minutes` are declared — but not initialised — before the call. The function fills them.

Three reference traps

1. A reference needs a variable

```
void addTen(int& n);

addTen(5);         // ERROR
addTen(a + b);     // ERROR
```

There is no variable for `n` to be a name of.

2. The `&` must be in the header

```
void addTen(int n) { n += 10; }
addTen(a);         // compiles, does
     nothing
```

3. The type must match exactly

```
void addTen(int& n);

double d = 5;
addTen(d);         // ERROR
```

A conversion would produce a temporary value, and a reference cannot name one.

Read the header first

`f(x)` tells you nothing. `void f(int& x)` tells you `x` may come back changed.

Which one should I write?

	Header	Use it when
by value	`void f(int n)`	the function only needs to *read* the value — the default choice
by reference	`void f(int& n)`	the function must *change* the caller's variable, or must return several answers

The habit

Start with by value. Add the `&` only when you have a reason — and when you do, say so in the function's name: `split`, `swapValues`, `readPair`.

A reader should never be surprised that a variable changed.

Your turn A

```
void addTen(int n)     { n += 10; }
void addTenRef(int& n) { n += 10; }

int main() {
    int a = 5, b = 5;
    addTen(a);
    addTenRef(b);
    cout << a << " " << b;
}
```

Your turn A

```
void addTen(int n)     { n += 10; }
void addTenRef(int& n) { n += 10; }

int main() {
    int a = 5, b = 5;
    addTen(a);
    addTenRef(b);
    cout << a << " " << b;
}
```

Output

```
5 15
```

Identical bodies, identical calls. The single `&` is the whole difference.

Your turn B

```
void grow(int& a, int b) {
    a = a * 2;
    b = b * 2;
}

int main() {
    int x = 3, y = 4;
    grow(x, y);
    grow(y, x);
    cout << x << " " << y;
}
```

Your turn B

```
void grow(int& a, int b) {
    a = a * 2;
    b = b * 2;
}

int main() {
    int x = 3, y = 4;
    grow(x, y);
    grow(y, x);
    cout << x << " " << y;
}
```

Output

```
6 8
```

First call: `x` doubles to 6, `y` untouched. Second call: `y` doubles to 8, `x` untouched — the second parameter is always a copy.

Function overloading

One name, several functions

```
int area(int side) {
    return side * side;
}

int area(int width, int height) {
    return width * height;
}

double area(double radius) {
    return 3.14159 * radius * radius;
}

cout << area(4)      << endl;   // 16
cout << area(2, 5)   << endl;   // 10
cout << area(1.0)    << endl;   // 3.14159
```

Overloading

Several functions may share a name as long as the compiler can tell the calls apart.

Why bother

The alternative is `areaSquare`, `areaRect`, `areaCircle`. One good name beats three clumsy ones — and `cout <<` has been overloaded for every type since Lecture 2.

What the compiler looks at

Distinguishes

- the number of parameters
- the types of the parameters
- their order: `f(int, double)` vs `f(double, int)`

Does not distinguish

- the return type
- the parameter *names*

```
int    value(int x);
double value(int x);      // ERROR: same parameters, only the return type differs
```

At the call `value(3)` the compiler sees only the arguments — it cannot know which return type you wanted.

When the compiler gives up

```
void show(int x)    { cout << "int "; }
void show(double x) { cout << "double "; }

show(5);        // int
show(5.0);      // double
show('A');      // int    (char -> int wins)

long k = 5;
show(k);        // ERROR: ambiguous
```

How a call is matched

An exact match wins. Otherwise the compiler looks for one conversion that is clearly better than the others.

Ambiguous

`long` → `int` and `long` → `double` are equally good, so there is no winner and the program does not compile.

Fix it by being explicit: `show(static_cast<int>(k));`

Overloads and default arguments can collide

```
int area(int side);
int area(int width, int height = 10);   // dangerous pair

cout << area(4);     // ERROR: ambiguous -- which one did you mean?
```

Both can accept one argument

`area(4)` matches the first function exactly, and matches the second with its default filled in. The compiler refuses to choose.

Use one or the other for the same “shape” of call — an overload set *or* a default argument, not both.

Your turn A

```
void show(int x)    { cout << "int "; }
void show(double x) { cout << "double "; }

int main() {
    show(5);
    show(5.0);
    show('A');
}
```

Your turn A

```
void show(int x)    { cout << "int "; }
void show(double x) { cout << "double "; }

int main() {
    show(5);
    show(5.0);
    show('A');
}
```

Output

```
int double int
```

`'A'` is a `char`: promoting it to `int` is a better match than converting it to `double`.

Your turn B

```
int biggest(int a, int b) {
    return (a > b) ? a : b;
}

int biggest(int a, int b, int c) {
    return biggest(biggest(a, b), c);
}

int main() {
    cout << biggest(3, 9) << " "
         << biggest(4, 11, 7);
}
```

Your turn B

```
int biggest(int a, int b) {
    return (a > b) ? a : b;
}

int biggest(int a, int b, int c) {
    return biggest(biggest(a, b), c);
}

int main() {
    cout << biggest(3, 9) << " "
         << biggest(4, 11, 7);
}
```

Output

```
9 11
```

The three-argument version does not repeat the comparison — it calls the two-argument one twice. That is how overloads should cooperate.

Wrap-up

What to take away

Shape and arguments

- A function is a contract: return type, name, parameter list.
- Declare before use — prototypes above `main`, definitions below.
- Arguments are matched by number, order and type, and converted to the parameter's type.
- Parameters and locals live only inside their own call.

Value, reference, overloading

- By value is the default: the function gets a copy and the caller is safe.
- `int&` makes the parameter another name for the caller's variable — use it to change it, or to return several answers.
- Overloads differ by parameter list, never by return type.

Questions?